

DSA STUDY BLUEPRINT SERIES

98. Merge Two Binary Search Trees

Merging Two BSTs into a Single Balanced BST

Section 09 • C++ Core Data Structures & Algorithms

Core Concepts & Visual Blueprint

Theoretical models and memory architectures

Key Principles

- **Problem:** Merge two trees to form a balanced BST.
- Step 1: Extract sorted inorder lists. Step 2: Merge sorted lists. Step 3: Build balanced BST.

Memory & Execution Blueprint

List1: [1, 3] + List2: [2, 4] $\Rightarrow$ Merged: [1, 2, 3, 4] $\Rightarrow$
balanced BST

C++ Implementation Reference

Standard code layout and syntax

```
// Extract list1 and list2 using inorder traversals.  
// Merge list1 and list2 into a single sorted list.  
// Reconstruct balanced BST recursively.
```

Algorithmic Complexity & Best Practices

Performance evaluations and critical guidelines

Performance Table

Aspect / Case	Complexity
Time Complexity	$O(N1 + N2)$
Space Complexity	$O(N1 + N2)$

Key Practices & Gotchas

- **Important:** Achieves optimal $O(N1 + N2)$ runtime.
- Verify boundary conditions (e.g. empty inputs, single elements).
- Optimize dynamic memory allocations to prevent leaks.

Dry Run & Step-by-Step Execution

Trace analysis on a sample input case

Execution Walkthrough

Let's trace BST constraint validation checks verifying node ranges `[Min, Max]`:

Active Node	Valid range limit	Comparison check	Recursive branch step
15 (Root)	$[-\infty, +\infty]$	True $(-\infty < 15 < +\infty)$	Recurse Left child with range $[-\infty, 15]$
10 (Left)	$[-\infty, 15]$	True $(-\infty < 10 < 15)$	Recurse Right child with range $[10, 15]$

Interview Variations & Optimization Pathways

Common follow-up problems and optimization strategies

Key Variations & Follow-up Questions

- **Inorder property:** BST Inorder traversals yield strictly increasing sorted lists.
- **Tree Balancing:** Balanced tree heights (AVL/Red-black) guarantee logarithmic search times.
- **Related LeetCode Problems:** #98 Validate Binary Search Tree, #700 Search in a Binary Search Tree, #108 Convert Sorted Array to Binary Search Tree.